U02 · Primeros pasos con Java

En la Unidad 1 aprendiste a pensar en pseudocódigo. Ahora toca traducir todo eso al idioma real que va a entender el ordenador: **Java**.

1. De tu código a un programa ejecutándose

Recuerda lo que vimos sobre compiladores, intérpretes y máquinas virtuales: Java compila tu código `.java` a **bytecode** (ficheros `.class`), y la JVM lo ejecuta. Para programar necesitas el **JDK** (Java Development Kit, incluye compilador, JVM y librerías); si solo quisieras *ejecutar* programas ya compilados, bastaría con el JRE.

> **No te compliques con esto a mano**
>
> En cursos antiguos había que configurar a mano variables del sistema (`PATH`, `JAVA_HOME`, `CLASSPATH`) para poder compilar y ejecutar desde la línea de comandos. **IntelliJ IDEA se encarga de todo esto por ti**: detecta o descarga el JDK que necesites, y compila y ejecuta con un solo clic. Tú concéntrate en aprender el lenguaje.

2. Anatomía de tu primer programa

```java
public class PrimerPrograma {
    public static void main(String[] args) {
        System.out.println("¡Hola Mundo!");
    }
}
```

Java es **totalmente orientado a objetos**: no puede existir código suelto fuera de una clase. Por eso todo programa empieza declarando una clase. Cosas que debes saber desde ya:

- **`public class PrimerPrograma`**: declara una clase pública. Si una clase es `public`, el **nombre del fichero `.java` debe coincidir exactamente** con el nombre de la clase (mayúsculas incluidas) — y solo puede haber una clase pública por fichero.

- **`public static void main(String[] args)`**: es el método por el que la JVM **empieza a ejecutar** tu programa. Tiene que escribirse exactamente así para que la JVM lo reconozca. Por convenio, los nombres de clase empiezan en mayúscula (`MiPrograma`), y si tienen varias palabras, cada una empieza en mayúscula (`MiPrimerPrograma`).
- **`System.out.println(...)`**: llama al método `println` del objeto `out` (de la clase `System`), que escribe el texto en pantalla y salta de línea. `System.out.print(...)` hace lo mismo pero sin saltar de línea.

3. Reglas de escritura en Java

- **Java distingue mayúsculas de minúsculas** (*case sensitive*): `nota`, `Nota` y `NOTA` son tres cosas distintas.
- **Cada sentencia termina en punto y coma `;`** (excepto el cierre de un bloque `}`).
- **Escritura libre**: puedes usar espacios, tabulaciones y saltos de línea donde quieras dentro de una instrucción — pero sigue siempre las convenciones de indentado, son las que hacen tu código legible.
- **Bloques de sentencias**: se agrupan entre llaves `{ }`.
- **Comentarios**, tres tipos:

```
// comentario de una línea
/* comentario
   de varias líneas */
/** comentario de documentación — lo usa la herramienta javadoc */
```

4. Identificadores: cómo nombrar las cosas

Un identificador es el nombre que le das a una variable, un método, una clase… Reglas:

- No puede empezar por un dígito, ni contener espacios o símbolos de operadores.
- No puede ser una palabra reservada del lenguaje (`if`, `class`, `true`…).
- No puede repetirse dentro del mismo ámbito.

Y las **convenciones** (no obligatorias por el compilador, pero sí por este módulo — código que no las sigue se considera mal escrito):

Elemento	Convención	Ejemplo
Variables y métodos	`camelCase`, empiezan en minúscula	`sueldoNeto`, `calcularTotal()`
Clases	`PascalCase`, empiezan en mayúscula	`CuentaBancaria`
Constantes	Todo en mayúsculas, palabras separadas por _	`IVA_GENERAL`

> ⚠ **Evita la ñ y las tildes en el código**
>
> Java permite técnicamente identificadores con ñ, tildes o ç (usa Unicode), pero **no lo hagas**: por compatibilidad entre editores, teclados y sistemas, la convención universal es ceñirse al alfabeto inglés en el código. Escribe los comentarios y los textos que se muestran al usuario en el idioma que quieras — los nombres de variables y clases, en inglés o en español sin acentos.

5. Tipos de datos primitivos

Java tiene 8 tipos primitivos, cada uno con un tamaño y un rango fijos (a diferencia del pseudocódigo, aquí sí importa el detalle exacto):

Tipo	Para qué sirve	Tamaño	Rango
`byte`	Enteros muy pequeños	1 byte	−128 a 127
`short`	Enteros cortos	2 bytes	−32.768 a 32.767
`int`	Enteros (el más usado)	4 bytes	−2.147.483.648 a 2.147.483.647
`long`	Enteros muy grandes	8 bytes	$\pm 9{,}2 \cdot 10^{18}$
`float`	Decimales, precisión simple	4 bytes	$\pm 3{,}4 \cdot 10^{38}$
`double`	Decimales, precisión doble (el más usado)	8 bytes	$\pm 1{,}7 \cdot 10^{308}$

Tipo	Para qué sirve	Tamaño	Rango
`char`	Un único carácter Unicode	2 bytes	'\u0000' a '\uffff'
`boolean`	`true` / `false`	—	`true` o `false`

> ⚠ **Java no comprueba desbordamientos**
>
> Si a un `short` con el valor máximo (32.767) le sumas 1, Java **no lanza ningún error** — el valor da la vuelta y pasa a −32.768 (se comporta de forma cíclica). Elegir un tipo demasiado pequeño para tus datos es una fuente de bugs silenciosos: en caso de duda, usa `int` o `long` para enteros, y `double` para decimales.

Además de estos 8 tipos primitivos, hay **tipos de datos compuestos** (u *objeto*) como `String` (texto) — los verás en detalle un poco más abajo, y con mucho más detalle en las unidades 4 y 5.

6. Variables, ámbito y constantes

Declarar una variable exige indicar su **tipo** y su **identificador**:

```
int edad;                   // declaración
edad = 16;                  // asignación posterior
double precio = 19.99;      // declaración + inicialización a la vez
float p1 = 1.5f, p2 = 2.0f; // varias variables del mismo tipo a la vez
```

Si no inicializas una variable, Java le da un **valor por defecto**: `0` para los tipos numéricos, '\u0000' para `char`, `false` para `boolean`, y `null` para tipos objeto. Aun así, es buena práctica inicializar siempre tus variables explícitamente.

Por ahora, todas las variables que declares serán **variables locales**: existen solo dentro del método (o del bloque `{ }`) donde las declaras, y dejan de existir al salir de él. Más adelante (unidad 4) verás los **atributos de clase**, que viven más tiempo.

Para declarar una **constante** (un valor que no debe cambiar nunca), usa la palabra clave `final`, y nómbrala en mayúsculas por convenio:

```
final double IVA = 0.21;
// IVA = 0.25;  // ERROR: no se puede reasignar una variable final
```

7. Operadores y precedencia

Categoría	Operadores
Aritméticos	+ - * / % (resto)
Incremento/decremento	++ -- (pre e in-fijo — ver nota)
Relacionales	< > <= >= == !=
Lógicos	&& (Y) || (O) ! (NO)
Asignación	= += -= *= /= %=

> **División entera vs. división real**
>
> `5 / 2` da **2**, no 2.5 — cuando divides dos enteros, Java trunca el resultado a entero. Si necesitas el resultado decimal, al menos uno de los dos operandos debe ser `double` o `float`: `5.0 / 2` sí da `2.5`. El operador `%` (módulo) te da el resto de una división entera: `5 % 2` es `1`.

Los operadores ++ y -- tienen dos formas, y la diferencia importa:

- **Postfijo** (x++): usa el valor de x en la expresión actual, **y después** lo incrementa.
- **Prefijo** (++x): incrementa x **primero**, y después usa el nuevo valor en la expresión.

Como en pseudocódigo, hay un **orden de precedencia** (de mayor a menor): paréntesis → operadores unarios (++, --, !) → multiplicación/división/módulo → suma/resta → relacionales → igualdad (==/!=) → && → \|\| → asignación. Ante cualquier duda, usa paréntesis.

8. Conversión de tipos: automática y casting

Java convierte automáticamente un tipo “más pequeño” a uno “más grande” cuando hace falta (por ejemplo, un `int` a `double`), sin que tengas que hacer nada — se llama **conversión**

implícita o *widening*. Pero si quieres ir al revés (de un tipo "más grande" a uno "más pequeño", con riesgo de perder información), tienes que forzarlo explícitamente con un **casting**:

```
double precio = 19.99;
int precioEntero = (int) precio;   // casting explícito: precioEntero vale 19 (se trunca, no se redondea)

int entero = 10;
double real = entero;              // conversión implícita: no hace falta casting
```

9. La clase `Math`

Para cálculos matemáticos más allá de `+ - * /`, Java incluye la clase `Math` (no hace falta importarla, está siempre disponible):

```
double potencia = Math.pow(2, 10);   // 2 elevado a 10
double raiz = Math.sqrt(81);         // raíz cuadrada
double absoluto = Math.abs(-5);      // valor absoluto
double alAzar = Math.random();       // decimal aleatorio entre 0.0 (incluido) y 1.0 (excluido)
double pi = Math.PI;                 // constante π
```

10. La clase `String`

El texto en Java se representa con objetos `String`. Aunque técnicamente es una clase (no un tipo primitivo), se usa tan a menudo que Java le da un trato especial: se puede escribir entre comillas dobles y concatenar con `+`.

```
String nombre = "Ada";
String saludo = "Hola, " + nombre + "!";   // concatenación con +

nombre.length();          // número de caracteres
nombre.charAt(0);         // el carácter en la posición 0 -> 'A'
nombre.toUpperCase();     // "ADA"
nombre.substring(1, 3);   // caracteres de la posición 1 a la 2 -> "da"
nombre.trim();            // quita espacios al principio y al final
```

> ⚠ **`==` no compara el contenido de un String**
>
> `nombre == "Ada"` compara si son **el mismo objeto en memoria**, no si tienen el mismo texto — y puede darte `false` incluso cuando el texto es idéntico. Para comparar el **contenido** de dos `String`, usa siempre `.equals(...)`:
>
> ```
> if (nombre.equals("Ada")) { ... }   // correcto
> if (nombre == "Ada") { ... }         // ¡evítalo! comportamiento poco fiable
> ```
>
> Es uno de los errores más comunes al empezar con Java, y el compilador no te avisa — simplemente el programa se comporta de forma inesperada.

11. Fechas y números muy grandes

- **Fechas**: el paquete moderno `java.time` (`LocalDate`, `LocalDateTime`) es la forma recomendada de trabajar con fechas hoy en día — las clases antiguas `Date` y `Calendar` siguen existiendo por compatibilidad, pero están consideradas obsoletas para código nuevo.

  ```
  import java.time.LocalDate;

  LocalDate hoy = LocalDate.now();
  LocalDate cumple = LocalDate.of(2008, 3, 15);
  ```

- **`BigInteger`** (`java.math.BigInteger`): cuando necesitas enteros más grandes de lo que `long` puede representar (por ejemplo, cálculos criptográficos o factoriales enormes), `BigInteger` no tiene límite de tamaño — a cambio de ser bastante más lento que los tipos primitivos.

  ```
  import java.math.BigInteger;

  BigInteger a = new BigInteger("12345678901234567890123456789O");
  BigInteger b = a.multiply(BigInteger.TWO);
  ```

12. Entrada y salida de datos

Ya conoces `System.out.print()` y `System.out.println()`. Hay una tercera opción cuando necesitas controlar el formato exacto de la salida: **`printf`**.

```
System.out.printf("El número es %05d%n", 33);      // "El número es 00033"
System.out.printf("Pi vale %.2f%n", Math.PI);        // "Pi vale 3,14"
```

Símbolo	Tipo de dato
`%d`	Entero (`int`, `long`)
`%f`	Decimal (`float`, `double`) — `%.2f` limita a 2 decimales
`%s`	Texto (`String`)
`%c`	Carácter

Para leer del teclado, se usa la clase **`Scanner`** (hay que importarla desde `java.util`):

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
int edad = sc.nextInt();
String nombre = sc.nextLine();
```

Método	Lee
`nextInt()` / `nextLong()` / `nextShort()` / `nextByte()`	Un entero
`nextDouble()` / `nextFloat()`	Un decimal
`nextLine()`	Una línea completa de texto
`next()`	La siguiente palabra (hasta un espacio)

> ⚠ **La trampa clásica de Scanner**
>
> `nextInt()` lee el número, pero **no consume el salto de línea** que queda después de que el usuario pulse Intro. Si justo después llamas a `nextLine()` esperando leer una línea nueva, en realidad capturas ese salto de línea sobrante vacío — el programa "se salta" la lectura. La solución es intercalar una llamada extra a `nextLine()` para descartar ese sobrante:
>
> ```
> int edad = sc.nextInt();
> sc.nextLine();              // descarta el salto de línea sobrante
> String nombre = sc.nextLine();  // ahora sí lee la línea de verdad
> ```

Scanner no tiene un método para leer directamente un `boolean` ni un único carácter (`char`) — hay que leerlos como texto y convertirlos tú mismo.

13. Sentencias de decisión

Ya viste `if/else` en pseudocódigo — en Java se escribe casi igual, pero la condición va entre paréntesis y el cuerpo entre llaves:

```
if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    System.out.println("Suspenso");
}
```

Para alternativas múltiples, además de encadenar `if/else if`, Java tiene **`switch`**, en dos versiones:

Clásica (cuidado con el `break` — sin él, la ejecución "cae" al siguiente `case`):

```java
switch (dia) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    default:
        System.out.println("Otro día");
}
```

Moderna (expresión `switch`, sin necesidad de `break` ni riesgo de "caída" entre casos):

```java
String nombreDia = switch (dia) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    default -> "Otro día";
};
```

14. Enumeraciones

Cuando una variable solo puede tomar un conjunto **cerrado y conocido** de valores (los días de la semana, los palos de una baraja, los estados de un pedido…), en vez de usar `String` o números "mágicos", Java te deja definir un **tipo enumerado**:

```java
enum DiaSemana { LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO }

DiaSemana hoy = DiaSemana.MIERCOLES;
if (hoy == DiaSemana.SABADO || hoy == DiaSemana.DOMINGO) {
    System.out.println("¡Es fin de semana!");
}
```

⭐ **Be the Code**: usar un `enum` en vez de comparar contra los textos `"LUNES"`, `"MARTES"`… hace que el compilador te avise si escribes mal un valor (`DiaSemana.LUNS` no compila), cosa que un `String` mal escrito jamás te va a avisar hasta que el programa ya esté fallando en producción.

15. Tipos de error en Java

Retomando la clasificación de la Unidad 1, así se ven en la práctica al programar en Java:

- **Errores de compilación**: `javac` (o IntelliJ, subrayándolo en rojo) te avisa antes de ejecutar nada — por ejemplo, olvidar un punto y coma o escribir mal una palabra reservada.
- **Errores en tiempo de ejecución**: el código compila bien, pero algo falla al ejecutarse con ciertos datos — leer una posición de un array que no existe, dividir un entero entre 0. Java los representa con **excepciones** (las estudiarás a fondo en la unidad 3).
- **Errores lógicos**: el programa compila y se ejecuta sin quejarse, pero el resultado no es el esperado. Son los que más tiempo de depuración exigen, porque no hay ningún aviso — solo un resultado incorrecto.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA1. Reconoce la estructura de un programa informático…	a) estructura de un programa · d) tipos de variables · e) crear y usar variables · f) constantes y literales · g) operadores · h) conversión de tipos · i) comentarios
RA4. Desarrolla programas utilizando objetos y clases…	j) uso de conjuntos y librerías de clases (las clases prefabricadas: `Math`, `String`, `Scanner`, fechas, `BigInteger`)
RA5. Realiza operaciones de entrada y salida de información…	a) consola para E/S · b) formatos de visualización · c) posibilidades de E/S del lenguaje

> **Una aclaración sobre esta tabla**
>
> La programación didáctica etiqueta esta unidad como “RA1 y RA4”, pero los criterios de evaluación que lista en su tabla corresponden en realidad a RA1 y a RA5 (no a RA4) — parece un desajuste de la propia tabla original. He mantenido aquí los RA que sí encajan con el contenido real de la unidad (RA1, RA4.j y RA5), sin alterar ni inventar el resto del documento.

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras